



CSE3433 SOFTWARE ARCHITECTURE

PROGRAMME: COMPUTER SCIENCE (SOFTWARE ENGINEERING)

SEMESTER II 2025/2026

GROUP 6

PROJECT TITLE:

SMARTLEARN: CLOUD-NATIVE LEARNING MANAGEMENT SYSTEM (LMS)

LECTURER: DR. MOHAMAD NOR HASAN

NO.	GROUP MEMBERS	MATRIC NO.
1.	PUTERI IREENE HAZWANI BINTI MUHAMAD HAIZAR	S75702
2.	NURAIN NAZIRAH BINTI ABDULLAH	S75415
3.	NURUL ALESSA BINTI MOHD NIZAM	S76278

TABLE OF CONTENT

1.0 Core services or modules.....	2
1.1 User Management Module.....	2
1.2 Course Management Module.....	2
1.3 Assignment Module.....	2
1.4 Grading Module.....	2
2.0 Communication between system components.....	3
2.1 The Data Exchange Flow.....	3
3.0. Basic user interface or API endpoints.....	4
3.1 Authentication Endpoints.....	4
3.1.1 Register User.....	4
3.1.2 Login User.....	4
3.2 User Management Endpoint.....	4
3.2.1 Get Users (Filter by Role).....	4
3.3 Course Management Endpoints.....	5
3.3.1 Course Module API.....	5
3.4 Frontend Pages Mapping (UI Flow).....	6
3.4.1 Register Page.....	6
3.4.2 Login Page.....	7
3.4.3 Administrator Dashboard.....	8
3.4.4 Lecturer Dashboard.....	9
3.4.5 Student Dashboard.....	10

1.0 Core services or modules

The SMARTLEARN prototype has been successfully broken down into independent functional modules within the Node.js/Express backend engine. This ensures that a bug in one feature won't bring down the rest of the application. The following core modules have been fully implemented in this phase of the prototype:

1.1 User Management Module

This module handles user onboarding, secure logins, and identity verification. It distinguishes between student and lecturer roles. When a user logs in successfully, the module generates a secure JSON Web Token (JWT) that the frontend saves to keep the user logged in safely.

1.2 Course Management Module

This module acts as the class workspace hub. It allows lecturers to create unique course profiles by generating an automatic, alphanumeric code (e.g., CSF3433). Students interact with this module by typing in that specific code, which instantly links them to the class and populates it onto their personal dashboard.

1.3 Assignment Module

This module manages assignment creation and submission activities. Lecturers can create assignments by providing instructions and submission deadlines. Students can upload assignment files through the system, and the module records submission details while updating assignment statuses accordingly.

1.4 Grading Module

This module handles student assessment and grading activities. Lecturers can review submitted assignments and assign grades to students. Once grading is completed, the system updates the grade records and displays the latest grading status and results on the student dashboard.

2.0 Communication between system components

Because our system is completely decoupled, the frontend (Vercel) and the backend (Render) cannot see each other's files directly. Instead, they communicate over the internet using a Synchronous HTTPS REST API connection model.

2.1 The Data Exchange Flow

1. **The Trigger:** A user clicks a button on the Vercel frontend interface (for example, clicking "Join Course" after typing in a class code).
2. **The Request:** The frontend uses JavaScript `fetch()` to bundle that data into a lightweight **JSON payload** and sends it across the web via an HTTPS POST request to our Render backend API URL.
3. **The Security Interception:** The Node.js server receives the request. Before touching the database, our custom `authMiddleware.js` file intercepts the request to verify the user's JWT security token.
4. **The Database Transaction:** Once approved, the backend route handler runs a secure query to insert or update rows inside our cloud database.
5. **The Response:** The database passes the result back to the Node.js server, which reformats it into a clean JSON success status response and sends it right back to the Vercel frontend. The browser reads the JSON data and updates the visual interface instantly without needing a full page reload.

3.0. Basic user interface or API endpoints

SMARTLEARN system exposes RESTful API endpoints that allow the frontend (Vercel) to communicate with the backend (Node.js/Express). These endpoints handle authentication, user data retrieval, and course management operations.

3.1 Authentication Endpoints

3.1.1 Register User

- **Method:** POST
- **Endpoint:** /api/register
- **Description:** Creates a new user account (student or lecturer).
- **Controller:** authController.register

3.1.2 Login User

- **Method:** POST
- **Endpoint:** /api/login
- **Description:** Authenticates user credentials and returns a JWT token for session management.
- **Controller:** authController.login

3.2 User Management Endpoint

3.2.1 Get Users (Filter by Role)

- **Method:** GET
- **Endpoint:** /api/users
- **Query Parameter (optional):**
 - role=student
 - role=lecturer
- **Description:** Retrieves list of users, optionally filtered by role.
- **Functionality:**

- Fetches data from MongoDB User collection
- Returns selected fields: userId, name, email, role

3.3 Course Management Endpoints

3.3.1 Course Module API

- **Base Endpoint:** /api/courses

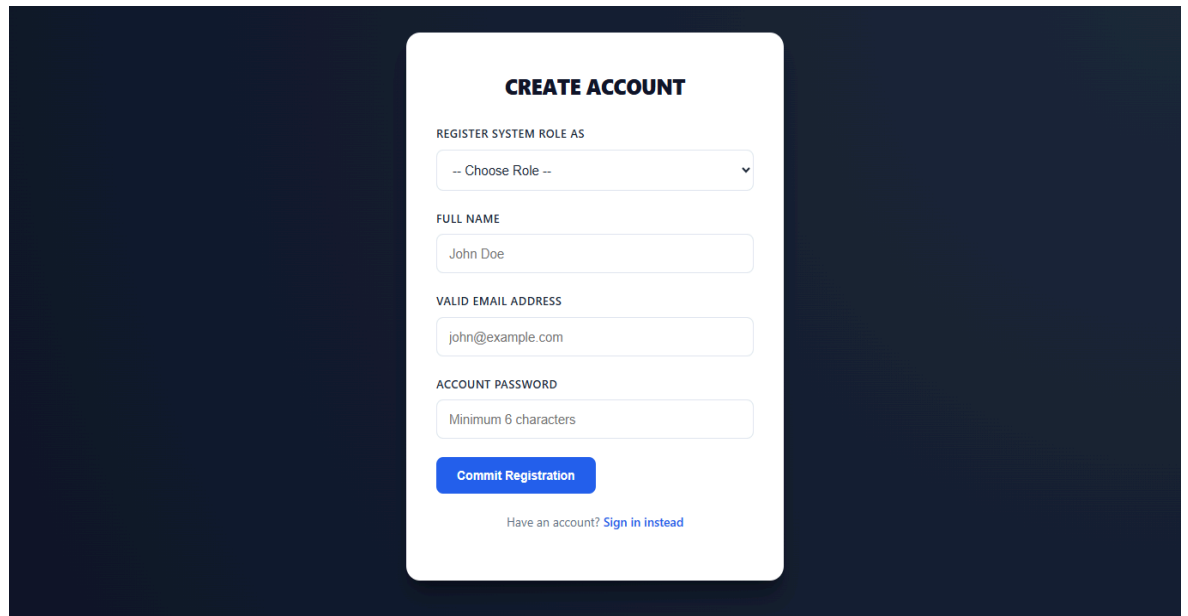
This module handles all course-related operations such as creation, joining, updating, and deletion.

- **Method:** Multiple (GET, POST, PUT, DELETE)
- **Controller:** courseController

3.4 Frontend Pages Mapping (UI Flow)

3.4.1 Register Page

- Uses:
 - POST /api/register
- Purpose:
 - User account creation (student/lecturer)



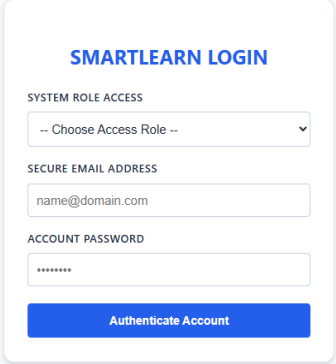
The image shows a registration form titled "CREATE ACCOUNT" centered on a dark blue background. The form is a white card with rounded corners. It contains the following fields and elements:

- REGISTER SYSTEM ROLE AS:** A dropdown menu with the text "-- Choose Role --" and a downward arrow.
- FULL NAME:** A text input field containing "John Doe".
- VALID EMAIL ADDRESS:** A text input field containing "john@example.com".
- ACCOUNT PASSWORD:** A text input field containing "Minimum 6 characters".
- Commit Registration:** A blue button with white text.
- Have an account? [Sign in instead](#)**: A link in blue text below the button.

Figure 3.4.1 : Registration Page

3.4.2 Login Page

- Uses:
 - POST /api/login
- Purpose:
 - User authentication and JWT generation



The image shows a login form titled "SMARTLEARN LOGIN" centered on a light blue background. The form is a white rounded rectangle with a thin blue border. It contains three input fields: a dropdown menu for "SYSTEM ROLE ACCESS" with the placeholder "-- Choose Access Role --", a text input for "SECURE EMAIL ADDRESS" with the placeholder "name@domain.com", and a password input for "ACCOUNT PASSWORD" with masked characters "*****". Below these fields is a blue button with the text "Authenticate Account".

Figure 3.4.2 : Login Page

3.4.3 Administrator Dashboard

- Uses:
 - GET /api/users
- Purpose:
 - Create, update, delete and view all subject

SMARTLEARN MASTER WORKSPACE (ADMIN)

ADMIN (ADMIN) [Sign Out](#)

Add New Academic Course Module

UNIQUE COURSE CODE

E.G. CSF3433

COURSE TITLE DESIGNATOR

e.g. Software Architecture Frameworks

SYLLABUS COURSE SPECIFICATIONS / DESCRIPTION

Enter course summaries...

ASSIGN DESIGNATED LECTURER ID

-- Select Lecturer --

Commit Data Record

Global System Catalog Index (Admin Master Privileges)

Index	Course Title Description	Subject Code	Lecturer ID	Administrative Actions
1	SOFTWARE ARCHITECTURE Intro course software architecture	CSE3433	ID: LMT004	Edit Delete
2	NETWORKING Intro of networks	CSA3234	ID: LMT005	Edit Delete
3	OOP coding	1234	ID: LMT001	Edit Delete
4	STATISTICS intro	SA78	ID: LMT004	Edit Delete
5	ENTREPRENEUR intro	MPU3223	ID: LMT005	Edit Delete

Figure 3.4.3: Administrator Dashboard Page

3.4.4 Lecturer Dashboard

- Uses:
 - /api/courses (course management APIs)
- Purpose:
 - Create, view, delete and update courses

The screenshot displays the SMARTLEARN LECTURER PORTAL interface. At the top, a dark blue header bar contains the text "SMARTLEARN LECTURER PORTAL" on the left and "ALESSA [LECTURER] Sign Out" on the right. The main content area is light blue and features two primary sections. The first section, titled "Add New Academic Course Module", contains three input fields: "UNIQUE COURSE CODE" (with the example "E.G. CSF3433"), "COURSE TITLE DESIGNATOR" (with the example "e.g. Software Architecture Frameworks"), and "SYLLABUS COURSE SPECIFICATIONS / DESCRIPTION" (with the placeholder "Enter course summaries..."). Below these fields is a blue "Commit Data Record" button. The second section, titled "Assigned Teaching Syllabus Modules", displays a list of modules. The first module shown is "1. OOP" with a green "Code: 1234" and the text "coding". Below this information are two buttons: a grey "Edit" button and a red "Drop" button.

Figure 3.4.4: Lecturer Dashboard Page

3.4.5 Student Dashboard

- Uses:
 - /api/courses
- Purpose:
 - Join courses using course code
 - View enrolled courses

Index	Course Title Description		Subject Code
1	SOFTWARE ARCHITECTURE Intro course software architecture	Enrolled ✓	CSE3433
2	NETWORKING intro of network	Enrolled ✓	CSA1234
3	OOP coding	Enrolled ✓	1234

Figure 3.4.5: Student Dashboard Page

SMARTLEARN frontend communicates with backend through REST API endpoints using JSON format over HTTPS. Each UI page interacts with specific backend routes depending on user role and functionality.